# 052400-1 VU Information Management and Systems Engineering (2025S)

## Milestone 1


Group 05

Student 1: Golovanov, Ilja, 12133820

Student 2: Togizbayev, Yernaz, 01429473


[*11.04.2025*], Vienna

# Contents

**1 Milestone 1** **3**
1.1 Team - Conceptual Modeling . . . 3
1.1.1 Describe the Application Domain . . . 3
1.1.2 Logical Design – ER Diagram in Chen Notation . . . 4
1.1.3 Relational Modeling – SQL CREATE Statements . . . 4
1.2 Individual - Student 1 . . . 5
1.2.1 Use Case Definition and Design . . . 5
1.2.2 Analytics Report . . . 6
1.2.3 NoSQL Design . . . 8
1.3 Individual - Student 2 . . . 10
1.3.1 Use Case Definition and Design . . . 10
1.3.2 Analytics Report . . . 11
1.3.3 NoSQL Design . . . 12

| Requirements | Realization |
|---|---|
| **Entities** | |
| 6- 9 Entities with key attribute & at least two other attributes | Office |
| | Client |
| | Employee |
| | Staff member |
| | Manager |
| | Boat |
| | Yacht |
| | Motorboat |
| | Catamaran |
| At least one weak entity | Rental |
| **Relationships** | |
| At least one unary (recursive) relationship | Manager supervises manager |
| At least one binary relationship (1:n) | Employee works in Office |
| | Client makes Rental |
| At least one binary relationship (m:n) | Staff Member maintains Boat |
| At least one IS-A (inheritance) relationship with 2+ child entities | Employee → Manager, Staff Member |
| | Boat → Yacht, Motorboat, Catamaran |

# 1. Milestone 1

## 1.1. Team - Conceptual Modeling

### 1.1.1. Describe the Application Domain

Our **boat** rental company operates through multiple **offices [street, country, ZIP, city]**, where **employees [first name, last name, birth date, mobile number, self-insurance number, email, street, country, ZIP, city, salary] work** in these locations. An **employee is** either a **manager [department, management level]** or a **staff member [work shift, is on duty]**. **Every employee works** at **exactly one office**. There is a hierarchy of **managers**, which means **one manager supervise many** other **managers** with lower management levels.

Customers are represented by **clients [first name, last name, birth date, email, street, ZIP, city, country, mobile number, captain license number]**. Customers can **make rentals [rental date, rental end date, payment status]**, which always **scheduled for** a **specific boat** and belong to **specific client**. A **rental** cannot exist without both a **client** and a **boat**.

**Boats [serial number, length, weight, seats, manufacturer, availability status, horsepower]** come in **different types**: **yachts [yacht name, has jacuzzi]**, **motorboats [engine type, fuel type]**, and **catamarans [max capacity, number of cabins]**. Each boat can be **maintained** by **several staff members**. Furthermore each **Boat belongs** to a **certain office**.

### 1.1.2. Logical Design – ER Diagram in Chen Notation

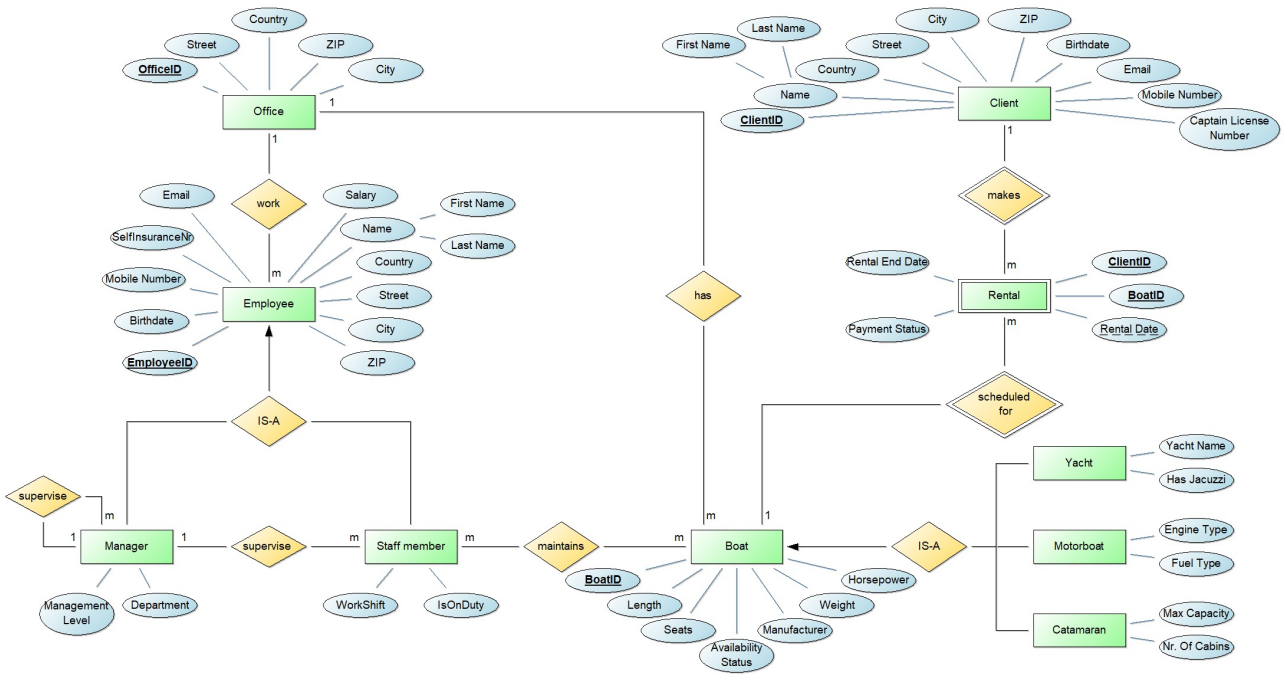



Figure 1: Boat Rental Diagram

### 1.1.3. Relational Modeling – SQL CREATE Statements

- Office: OfficeID (PK), Country, City, ZIP, Address
- Employee: EmployeeID (PK), OfficeID(FK), FirstName, LastName, Street, ZIP, City, Country, Birthdate, Email, MobileNr, SelfInsuranceNr, Salary
- Manager: ManagerID (PK, FK), Department, ManagementLevel, SupervisorID (FK $\rightarrow$ ManagerID)
- Staff: StaffID (PK, FK), WorkShift, IsOnDuty
- Client: ClientID (PK), FirstName, LastName, Street, ZIP, City, Country, Birthdate, Email, MobileNr, CaptainLicenseNumber
- Boat: BoatID (PK), Length, Seats, Manufacturer, AvailabilityStatus, Weight, Horsepower, OfficeID (FK)
- Yacht: YachtID (PK, FK), YachtName, HasJacuzzi
- Motorboat: MotorboatID (PK, FK), EngineType, FuelType
- Catamaran: CatamaranID (PK, FK), NrOfCabins, MaxCapacity
- Rental: ClientID (PK, FK), BoatID (PK, FK), RentalDate (PK), RentalEndDate, PaymentStatus
- Supervises: ManagerID (PK, FK), StaffID (PK, FK)
- Maintains: StaffID (PK, FK), BoatID(PK, FK)

In our relational model, we used a SupervisorID column in the Manager table to represent that one manager can supervise another. This is a small difference from the ER diagram, where this was shown as a separate relationship. We chose this way because it matches the 1-to-many relationship and makes it easier to query who supervises whom.

## 1.2. Individual - Student 1

Student 1: Golovanov, Ilja, 12133820

### 1.2.1. Use Case Definition and Design

#### Textual Description

*Weak entity*

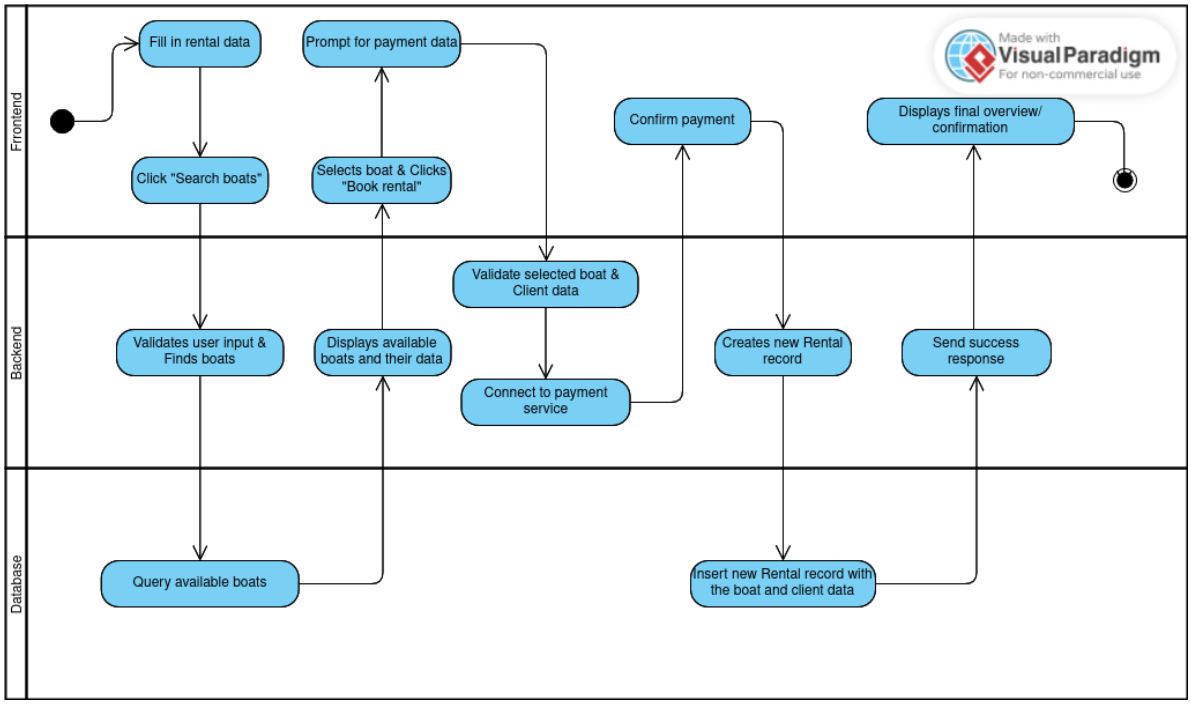
**Use Case Name:** Boat is being booked for a client
**Trigger:** A client chooses to book a boat online and fills out the forms.
**Preconditions:** The client is logged in.
**Main Flow:**

1. Client fills in data for a rental
   (a) Client enters the desired date for their rental.
   (b) Client enters the desired destination (office) of their rental.
   (c) Client enters if they need a skipper or not.
   (d) Client enters optionally what type of boat they want.
2. Backend validates the data and executes database query.
3. Database queries available boats corresponding with desired data.
4. Backend gets the data and displays it to the client.
5. Client sees a list of boats, chooses one and clicks "Book rental".
6. Client fills in payment information.
7. Backend validates the data and compares the desired outcome to the clients information.
8. Backend connects to the payment service.
9. Client confirms payment.
10. A new Rental record is being inserted into the database.
11. The new Rental record is linked with the user and the desired boat during insertion.
12. Backend summarizes all data and a confirmation with the summary is being presented to the client.

**Postconditions:** A new rental (weak entity) exists in the system and is linked to a client and to a boat.
**Entities:**

- Client $\rightarrow$ Entity needed for weak-entity.
- Boat $\rightarrow$ Parent-entity needed for weak-entity.
- Rental $\rightarrow$ Weak-entity depended on client and boat.

### Graphical Representation




## 1.2.2. Analytics Report

### Concept

**Use Case:** Client books a Rental for a boat
**Filter Field:** Boat.OfficeID.City (@desiredCity)
**Entities Involved:**

- Rental
- Boat
- Office
- Client

**SQL Query:**

```sql
SET @desiredCity = 'Dubrovnik';
SET @desiredStart = '2025-07-01';
SET @desiredEnd   = '2025-07-05';

SELECT
  b.BoatID,
  b.Length,
  b.Seats,
  b.Manufacturer,
  b.Weight,
  b.Horsepower
FROM Boat b
JOIN Office o
  ON b.OfficeID = o.OfficeID
WHERE o.City = @desiredCity
  AND b.AvailabilityStatus = 'Available'
  AND NOT EXISTS (
    SELECT 1
    FROM Rental r
    WHERE r.BoatID = b.BoatID
      AND r.RentalDate < @desiredEnd
      AND r.RentalEndDate > @desiredStart
  );
```

Listing 1: SQL Query for 'Booking rental' Use Case

Here would be a query, that checks which boats are currently available at a certain city. This is executed through looking up the city of the office and comparing that to the boats that office has currently available. It also checks for maintenance status of the boats and does not query them.

## Proof of Concept

To proof, that the use-case works and that the output of the above query changes, we perform the following:

**Step 1**: Create the ground structure of the database by executing the "Group05_Createtable.sql" file. This creates all the necessary tables. The concrete entities we need for the "book rental" use-case are listed above: **Office, Client, Boat, Rental**.

**Step 2**: Insert the dummy data needed for our use-case.

**Step 3**: We execute the above defined query to see our initial state and see the following output in the console:

```
+--------+--------+-------+--------------+--------+------------+
| BoatID | Length | Seats | Manufacturer | Weight | Horsepower |
+--------+--------+-------+--------------+--------+------------+
| B201   |     22 |     6 | MotorStar2   |   1100 |         70 |
| B300   |     26 |    10 | CataLite1    |   1300 |         50 |
+--------+--------+-------+--------------+--------+------------+
```

One can see that the office of the desired city "Dubrovnik", has currently two boats available.

**Step 4**: Now we execute the use-case "Book rental" process.

**Step 5**: We execute the defined query again to look if the output has changed.

```
+--------+--------+-------+--------------+--------+------------+
| BoatID | Length | Seats | Manufacturer | Weight | Horsepower |
+--------+--------+-------+--------------+--------+------------+
| B201   |     22 |     6 | MotorStar2   |   1100 |         70 |
+--------+--------+-------+--------------+--------+------------+
```

The output of the query shows only one available boat left. It correctly identifies, that the boat with the BoatID "B300" was booked and is not available anymore.

### 1.2.3. NoSQL Design

#### Design Overview

The weak entity Rental is dependend on the entities Boat and Client and by assumption always referenced in combination with either. Therefore I decided to include the Rental entity completely in the Client entity for the NoSQL design. In the rental record itself is then the boat (through the BoatID) referenced.
**Entities involved:**

- Boat
- Client
- Rental

**Json structure:**

```
{
  "_id": "C123",
  "firstName": "Max",
  "lastName": "Mustermann",
  "street": "Bergergasse 8-10/7/15",
  "zip": "1080",
  "country": "Austria",
  "city": "Vienna",
  "birthdate": "1990-01-01",
  "email": "max.mustermann@example.com",
  "mobileNumber": "+43-555-123-4567",
  "captainLicenseNumber": "CAPT-123456",
  "rentals": [
    {
      "boatID": "B100",
      "rentalDate": "2025-05-01",
      "rentalEndDate": "2025-05-03",
      "paymentStatus": "PAID"
    },
    {
      "boatID": "B234",
      "rentalDate": "2025-10-12",
      "rentalEndDate": "2025-10-15",
      "paymentStatus": "UNPAID"
    }
  ]
}
```

## Expected Execution and Possible Changes

**Expected Execution:**
The "Book Rental" use-case would only needs to do one write operation on one collection, the client. More concretely edit the "rentals" array of the specific client in question, making it very simple. In case of looking up currently available boats at a certain location, the boat collection would be checked for the current availability status and all the rentals from all the clients would be compared. This on the other hand, needs more operations.

**Possible changes:**
If searching for rentals becomes more frequent, it might be better to make a collection "rentals" on its own. The entries would then reference the client and boat respectively. This also would be better if suddenly clients book many rentals, containing a huge array. Although this seems to be less likely.

## Five Rules of Thumb

1. Favor embedding unless there is a good reason not to:
   → Rental is embedded into the client.

2. The need to access an object on its own is a compelling reason not to embed it:
   → Rentals are accessed mostly through the context of the client or the boat, so it should not be a problem to keep the rentals embedded.

3. High-cardinality arrays are a compelling reason not to embed:
   → It seems to be unlikely that a client would have a large number of rentals.

4. Consider the write/read ratio of a collection/document when denormalizing:
   → Writing operations are only happening during a booking, which is a good. The reading operations I am not sure, especially by looking up available boats with a certain filter applied.

5. Structure your data to match the ways that your application queries and updates it:
   → The update happens very easy, although the specific query for an available boat seems to be less performative. If those queries end up being more frequent than adding a rental after a booking, this would have to change. Maybe embedding it into the boat would then be optimal.

## 1.3. Individual - Student 2

Student 2: Togizbayev, Yernaz, 01429473

### 1.3.1. Use Case Definition and Design

#### Textual Description

*IS-A relationship*

**Use Case Name:** Manager adds a new staff member
**Trigger:** A manager opens the employee creation form and chooses to add a new staff member
**Preconditions:** Manager is logged in/authenticated and assigned to an office
**Postconditions:** A new employee exists in the system with a "staff" subtype, and is linked to an office
**Main Flow:**
(1) Fill in staff information
(1.1) Manager enters staff member's personal and work-related details
(1.2) Backend validates the form data and checks that the office exists
(1.3) A new employee record is inserted into the database
(1.4) A corresponding staff record is inserted and linked to the employee
(1.5) Staff member is connected to the selected office
(1.6) A supervision record is created linking the manager to the newly hired staff member
(2) Confirmation is shown to the manager
**Entities:** Manager → IS-A subtype of Employee
Staff → IS-A subtype of Employee
Employee → parent of Staff and Manager
Office → entity outside of the IS-A relationship
**Clarification:** In this system, the action of hiring is represented by the Supervises relationship. When a manager supervises a staff member, it implies that the manager was responsible for hiring them.

#### Graphical Representation

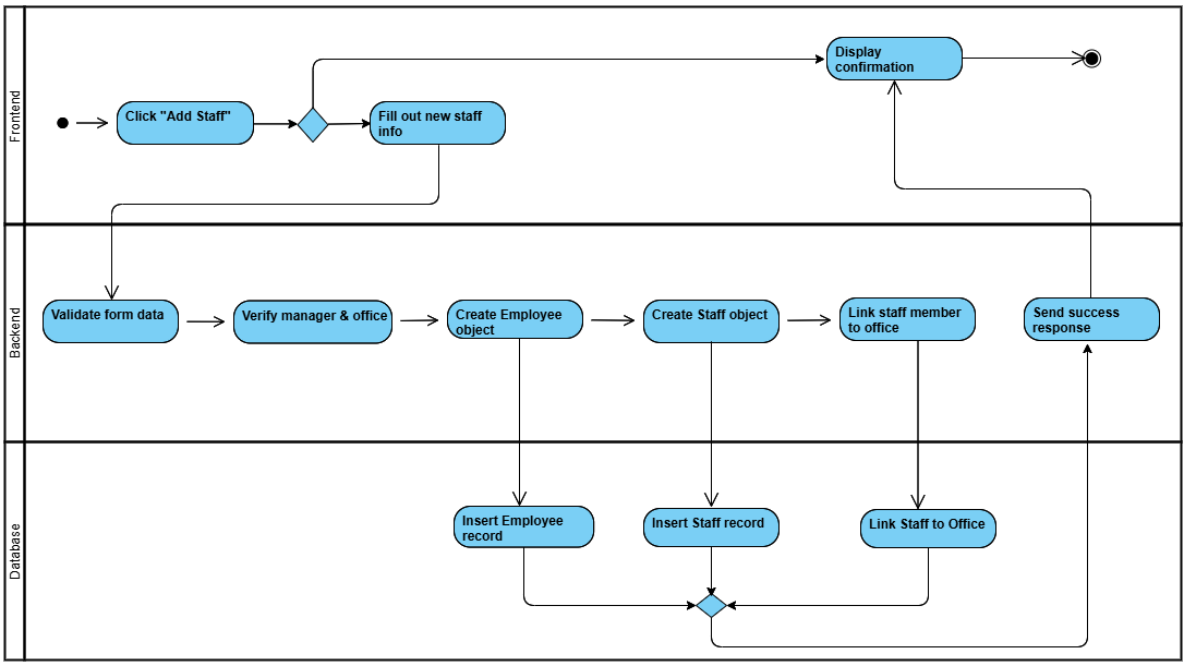



Figure 2: UML Activity Diagram (of simple use case)

### 1.3.2. Analytics Report

#### Concept

**Use Case:** Manager adds a new staff member and supervises them
**Filter Field:** Manager.ManagerID (e.g. WHERE m.ManagerID = 'M1')
**Entities Involved:**

- Office
- Employee
- Staff
- Manager
- Supervises

**SQL Query:**

```sql
SELECT
    s.StaffID,
    se.FirstName AS StaffFirstName,
    se.LastName AS StaffLastName,
    s.IsOnDuty,
    o.City,
    o.Country,
    CONCAT(me.FirstName, ' ', me.LastName) AS ManagerName
FROM Supervises sup
JOIN Staff s ON sup.StaffID = s.StaffID
JOIN Employee se ON s.StaffID = se.EmployeeID
JOIN Employee me ON sup.ManagerID = me.EmployeeID
JOIN Manager m ON m.ManagerID = me.EmployeeID
JOIN Office o ON se.OfficeID = o.OfficeID
WHERE m.ManagerID = 'M2';
```

Listing 2: SQL Query for 'New Staff Member Hiring' Use Case

This query is filtered by a specific manager (ManagerID = 'M2') and shows all staff members that this manager supervises. The result includes the details of the staff member and office information and changes each time the manager adds a new staff member under their supervision.

#### Proof of Concept

The aim is to show that the result of the query changes after performing the use case.
**Step1:** Execute all the necessary queries from 'Group05_Createtable.sql' to create tables for our use case. For our use case, these are: Office, Employee, Manager, Staff, Supervises.
**Step 2:** Insert dummy data into our tables from Step 1.
**Step 3:** Now we execute our SQL Query for filtering:

| StaffID varchar | StaffFirstName varchar | StaffLastName varchar | IsOnDuty tinyint | City varchar | Country varchar | ManagerName varchar |
|---|---|---|---|---|---|---|
| S1 | Jane | Doe | 1 | Mykonos | Greece | Anna Smith |
| S2 | Emily | Brown | 0 | Mykonos | Greece | Anna Smith |

Figure 3: Use Case Filter before Hiring new Staff Member

The data filtered by ManagerID 'M2',which belongs to the HR manager 'Anna Smith'. As we can see, for now she has 2 employees which she supervises. Now let's perform 'hiring' from out use case and re-run the filtering query.
**Step 4:** Add new Staff Member and place under 'M2' supervision.
**Step 5:** Re-run the query.

| StaffID varchar | StaffFirstName varchar | StaffLastName varchar | IsOnDuty tinyint | City varchar | Country varchar | ManagerName varchar |
|---|---|---|---|---|---|---|
| S1 | Jane | Doe | 1 | Mykonos | Greece | Anna Smith |
| S2 | Emily | Brown | 0 | Mykonos | Greece | Anna Smith |
| S8 | Lena | Karras | 1 | Mykonos | Greece | Anna Smith |

Figure 4: Use Case Filter before Hiring new Staff Member

The proof of concept demonstrates that the analytics report correctly reflects the use case. Initially, Manager 'M2' supervises two staff members (S1 and S2). After simulating the use case — where 'M2' adds a new staff member (S8) — the query updates accordingly to show three supervised staff. The screenshots before and after execution confirm that the system reacts as expected to data manipulation operations.

### 1.3.3. NoSQL Design

#### Design Overview

In my NoSQL design, each manager is stored as one document, and their supervised staff members are included directly inside it. Since the manager is the one who hires the staff member in my use case, I decided to keep the staff information together with the manager's data. This structure makes it easy to see which staff member belong to each manager, and it keeps all the related information together in one place.

**Entities involved in this use case:**

- Manager
- Staff
- Employee
- Office
- Supervises (relationship)

**JSON structure:**

```
{
  "manager_id": "M2",
  "name": {
    "first": "Anna",
    "last": "Smith"
  },
  "department": "HR",
  "management_level": "Senior",
  "office": {
    "office_id": "O1",
    "city": "Mykonos",
    "country": "Greece"
  },
  "supervisor_id": "M1",
  "supervised_staff": [
    {
      "staff_id": "S1",
      "name": {
        "first": "Jane",
        "last": "Doe"
      },
      "work_shift": "Morning",
      "is_on_duty": true
    },
    {
      "staff_id": "S2",
      "name": {
    "first": "Emily",
        "last": "Brown"
      },
      "work_shift": "Evening",
      "is_on_duty": false
    },
    {
      "staff_id": "S8",
      "name": {
        "first": "Lena",
        "last": "Karras"
      },
      "work_shift": "Evening",
      "is_on_duty": true
    }
  ]
}
```

Listing 3: MongoDB Data Structure Design

## Expected Execution and Possible Changes

**Expected Execution:**

In this use case, the manager hires a new staff member. So when this happens in the NoSQL version, only one write operation is needed: we just add the new staff member to the list of supervised staff under that manager.

This is helpful because everything we need — the manager and their staff — is already stored together. We don't need to update anything in other collections.

**Possible Changes:**

If we later end up hiring lots of staff, or need to access staff members separately more often, we could change the structure to store staff in their own collection and just reference them by ID.

If there are lots of searches based on things like work shift or duty status, we could also add indexes to those fields inside the staff array to make the queries faster.

## Five Rules of Thumb

1. Favor embedding unless there is a good reason not to:
   → Staff embedded inside the manager because they're directly connected and always shown together.

2. The need to access an object on its own is a compelling reason not to embed it:
   → In this case, staff are usually accessed through their manager, so it's fine to keep them embedded.

3. High-cardinality arrays are a compelling reason not to embed:
   → In this use case, each manager supervises a small team. If it grows, referencing could be used instead.

4. Consider the write/read ratio of a collection/document when denormalizing:
   → My design works well for reads, since all info is in one place. There aren't many writes happening at once, so this structure fits well.

5. Structure your data to match the ways that your application queries and updates it:
   → Since the manager is the one doing the hiring, and we often want to see who they supervise, storing the data like this makes sense for my use case.